Clinical Decision Support System for MIMIC IV Discharge Notes

By Pratik Tribhuwan & Sonal Phatige Sanjay

1. Executive Summary

This project implements a locally run Clinical Decision Support System (CDSS) for querying patient-level information from MIMIC-IV discharge summaries and related structured hospital data. The system uses a Retrieval-Augmented Generation (RAG) architecture: it ingests clinical records, preprocesses discharge notes into sections and chunks, extracts clinical entities, embeds those chunks, stores them in a vector database, retrieves relevant evidence for a clinician query, and generates a grounded answer using a local language model [1], [2].

The final application includes a FastAPI backend, a React + TypeScript frontend, a legacy Streamlit demo, evaluation scripts, and a notebook-based evaluation walkthrough. The system is designed for local execution so that sensitive clinical data and model inference remain on the user machine.

2. Project Motivation

Clinical records contain large amounts of useful patient information, but much of it is buried in long discharge notes and fragmented structured tables. Clinicians often need quick answers to questions such as:

- What was the discharge diagnosis?
- Which medications was the patient discharged on?
- What procedures were performed?
- What allergies or relevant history were documented?

The goal of this project is to reduce the time needed to locate this information while preserving traceability. Instead of asking a language model to answer from memory, the system retrieves patient-specific text chunks and generates an answer only from those retrieved sources.

The application is intentionally scoped to chart-grounded question answering. In the default workflow, it does not perform web search and does not provide general medical advice beyond the retrieved patient record. A user first selects a hospital admission, then asks questions that are answered from that stay's discharge notes and related structured data.

3. Objectives

The project had five main objectives:

1. Build an end-to-end RAG pipeline for clinical discharge notes.
2. Integrate structured MIMIC-IV data such as diagnoses, medications, procedures, admissions, and demographics.
3. Provide a usable clinician-facing interface for patient overview and question answering.
4. Evaluate retrieval and generation quality against held-out clinical questions.
5. Keep the default stack local-first to avoid external API dependencies for sensitive data.

4. Dataset

The system uses de-identified MIMIC-IV data, a credentialed critical care dataset commonly used for clinical informatics research [1]. The files used in this project include:

- `admissions.csv` for admission metadata.
- `patients.csv` for demographics such as gender and anchor age.
- `discharge.csv` for discharge summary text.
- `diagnoses_icd.csv` and `d_icd_diagnoses.csv` for ICD diagnosis codes and descriptions.
- `procedures_icd.csv` and `d_icd_procedures.csv` for ICD procedure codes and descriptions.
- `prescriptions.csv` for medication orders, dose fields, routes, and timestamps.

The ingestion layer loads a configurable number of admissions, filters note and medication tables to the selected `hadm_id` values, and builds `PatientRecord` objects keyed by admission ID.

5. System Architecture

The core pipeline is implemented as modular Python components:

- `DataIngestionModule`: loads MIMIC-IV admissions, notes, diagnoses, procedures, medications, and demographics.
- `PreprocessingModule`: cleans discharge summaries, detects clinical sections, and creates overlapping text chunks.
- `NERModule`: extracts clinical entities using SciSpaCy and section-based categorization [3].
- `EmbeddingModule`: generates local embeddings using Ollama and `nomic-embed-text`.
- `VectorStoreModule`: stores embedded chunks and metadata in ChromaDB for similarity search.
- `GenerationModule`: creates grounded responses from retrieved patient context using a local Ollama model.

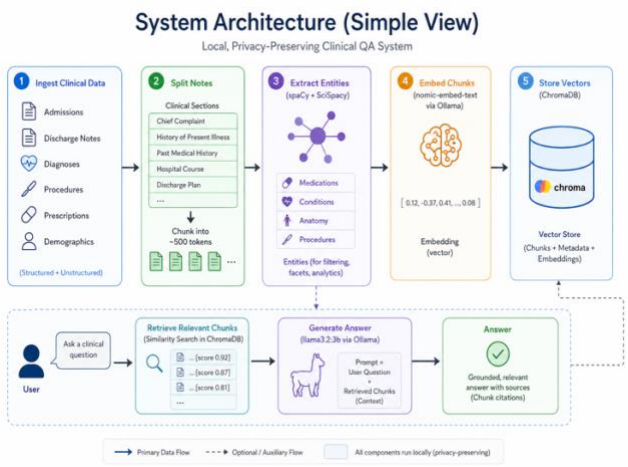

Figure 1. System architecture from the project presentation

The diagram summarizes the local RAG workflow from MIMIC-IV ingestion through preprocessing, embeddings, ChromaDB retrieval, local LLM generation, and the user-facing application.

5.1 Data Ingestion

`DataIngestionModule` reads the MIMIC-IV CSV files and combines structured and unstructured data into patient admission records. Each record contains admission/discharge metadata, discharge notes, ICD-backed diagnoses, procedures, medications, and demographics.

Large files such as discharge notes and prescriptions are streamed in chunks to avoid loading unnecessary rows into memory. This is important because MIMIC-IV files can be large even when the demo is configured to load only a small number of admissions.

5.2 Preprocessing

`PreprocessingModule` splits discharge summaries into recognizable clinical sections such as:

- Chief complaint
- History of present illness
- Past medical history
- Medications on admission
- Discharge medications
- Allergies
- Physical exam
- Pertinent results
- Discharge diagnosis
- Discharge instructions

Each section is cleaned and divided into overlapping word-level chunks. Chunk metadata includes the patient ID, admission ID, section name, chunk ID, and chunk index. This structure allows the retrieval layer to return not only relevant text but also where the text came from.

5.3 Named Entity Recognition

`NERModule` uses the SciSpaCy `en_core_sci_sm` model to extract clinical entity spans. Because the default model returns broad entity labels, the implementation also uses section-based heuristics to bucket extracted terms into:

- Medications
- Diseases
- Anatomy
- Procedures

These extracted entities are used in the patient overview interface and stored as vector metadata.

5.4 Embeddings

`EmbeddingModule` generates text embeddings using Ollama with the `nomic-embed-text` model. The system uses separate query and document prefixes so that text chunks and user questions are embedded in a retrieval-friendly format.

5.5 Vector Storage and Retrieval

`VectorStoreModule` stores embedded chunks in a persistent ChromaDB collection using cosine similarity. Each stored chunk includes:

- Chunk text
- Admission ID
- Patient ID
- Clinical section
- Extracted entity metadata

Retrieval can be global or scoped to a single `hadm_id`. The deployed dashboard uses patient-scoped retrieval, which significantly improves relevance because many discharge summaries contain semantically similar clinical language.

5.6 Design Considerations

Two design decisions were especially important.

1. The system uses single-patient scoping for the dashboard QA workflow. Clinical queries such as "What medications was the patient discharged on?" are highly reusable across patients. Without a patient filter, the vector store may retrieve semantically similar chunks from the wrong admission. Scoping retrieval by `hadm_id` keeps the answer grounded in the selected patient's chart.

2. The system evaluates the number of retrieved chunks rather than assuming more context is always better. Retrieving too few chunks can miss the correct section, while retrieving too many chunks can add irrelevant context, slow responses, and dilute precision. The notebook experiments examine recall and precision across values of `k`. On the current evaluation set, `k = 5` is a strong practical default for patient-scoped RAG because increasing to `k = 10` usually does not meaningfully improve whether the correct passage is found, but it does add more irrelevant text to the prompt.

Mean Recall (R) and Precision (P) per strategy across 20 queries

k	RAG Global R	RAG Scoped R	Keyword R	RAG Global P	RAG Scoped P	Keyword P
1	0.000	0.750	0.650	0.000	0.750	0.650
2	0.000	0.800	0.800	0.000	0.400	0.400
3	0.050	0.800	0.800	0.017	0.267	0.267
5	0.100	0.950	0.800	0.020	0.190	0.160
7	0.150	0.950	0.850	0.021	0.136	0.121
10	0.200	1.000	0.900	0.020	0.100	0.090

Figure 2. Recall and precision versus number of retrieved chunks

This notebook figure supports the design choice to tune `k` instead of arbitrarily retrieving more context. Higher `k` generally improves recall, but precision and prompt efficiency must also be considered.

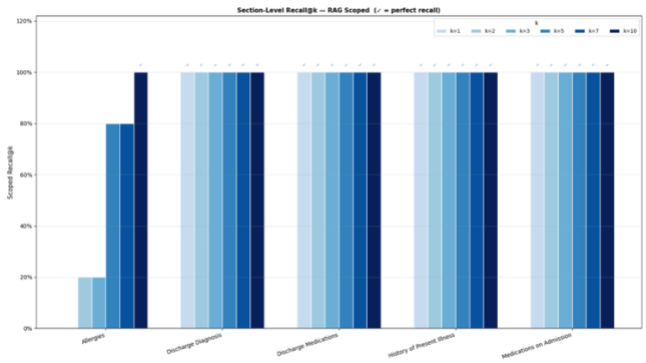

Figure 3. Section-level recall across retrieval depths

The figure shows that retrieval behavior varies by clinical section, which motivated attention to section-aware preprocessing and evaluation.

5.7 Generation

`GenerationModule` uses a local Ollama model, `llama3.2:3b`, to generate grounded answers. For RAG questions, the model receives retrieved clinical context and is instructed to answer using that context. The module also supports raw prompt generation for dashboard summaries.

5.8 Advanced Retrieval Techniques Explored

Beyond the main scoped RAG pipeline, the project explored advanced retrieval ideas documented in the evaluation notebook and related branch work:

- RAG-Fusion, where multiple rewritten versions of a query are retrieved and merged to improve coverage [4]. In this project, it did not solve the wrong-patient problem when retrieval was global because the rewritten queries were often still semantically similar across patients.
- Reciprocal Rank Fusion (RRF), where ranked result lists are combined using reciprocal rank scoring [5].
- Deterministic section routing, where queries are mapped to likely clinical note sections such as allergies, discharge diagnosis, or discharge medications. This worked well when the query matched a known pattern, but it was fragile for free-text questions and would require ongoing rule maintenance.
- Hybrid vector and keyword retrieval, which combines semantic search with lexical matching for exact clinical terms. It showed limited improvement once retrieval was already scoped to one admission at the current corpus scale.
- Patient matching prototypes based on entity overlap for cross-patient search experiments.

These methods are treated as exploratory unless they are validated against the held-out evaluation set. The production-facing dashboard keeps the simpler, more reliable patient-scoped vector pipeline and prioritizes data quality issues, such as very short allergy lines, over more complex fusion logic.

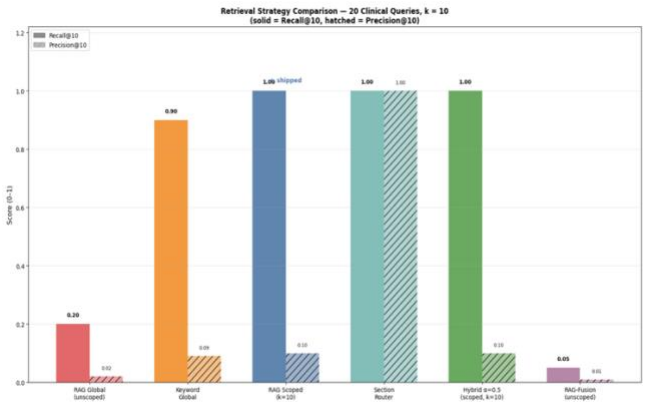

Figure 4. Retrieval strategy comparison. The notebook compares baseline and advanced retrieval approaches, including scoped RAG and experimental advanced strategies.

6. Application Layer

6.1 FastAPI Backend

The backend is implemented with FastAPI. On startup, it builds the RAG pipeline in a background thread so the server can accept requests immediately. Clients poll `/status` until the pipeline is ready. Routes that depend on indexed data return `503` until the pipeline is ready.

6.2 React Frontend

The primary user interface is a React 18 + TypeScript single-page application built with Vite and Tailwind CSS. It includes:

- A pipeline status banner with build logs.
- A patient selector.
- An overview tab with demographics, diagnosis and medication counts, procedure summaries, NER entity charts, and an AI-generated clinical brief.
- A medications tab with medication lists, route charts, frequency charts, and timeline-style displays.
- A diagnoses tab with ICD-backed diagnosis descriptions and procedure lists.
- A chat tab for patient-scoped RAG question answering with source evidence.

The frontend communicates with the FastAPI backend over JSON HTTP requests.

6.3 Streamlit Demo

The repository also contains a Streamlit app for quick single-process experiments. This is useful for lightweight demonstrations, while the React + FastAPI stack is the main application path.

7. Evaluation

The project includes both code-based and notebook-based evaluation.

7.1 Retrieval Evaluation

The retrieval evaluation compares global RAG, patient-scoped RAG, and keyword retrieval. The held-out evaluation set contains 20 clinical questions across 10 patients, covering sections such as discharge diagnosis, discharge medications, medications on admission, allergies, and history of present illness.

The main finding is that patient scoping is essential. Without an admission filter, semantically similar notes from other patients can dominate the vector search results. When retrieval is constrained to the selected admission, recall improves substantially.

	Query ID	Section	Relevant	RAG P	RAG R	Scoped P	Scoped R	KW P	KW R
0	Q01	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.000000	0.000000
1	Q02	discharge_medications	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
2	Q03	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
3	Q04	medications_on_admission	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
4	Q05	allergies	1	0.000000	0.000000	0.200000	1.000000	0.000000	0.000000
5	Q06	discharge_medications	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
6	Q07	allergies	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
7	Q08	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
8	Q09	allergies	1	0.000000	0.000000	0.200000	1.000000	0.000000	0.000000
9	Q10	history_of_present_illness	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
10	Q11	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.000000	0.000000
11	Q12	discharge_medications	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
12	Q13	allergies	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
13	Q14	history_of_present_illness	1	0.200000	1.000000	0.200000	1.000000	0.200000	1.000000
14	Q15	medications_on_admission	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
15	Q16	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
16	Q17	history_of_present_illness	1	0.200000	1.000000	0.200000	1.000000	0.200000	1.000000
17	Q18	discharge_diagnosis	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000
18	Q19	allergies	1	0.000000	0.000000	0.000000	0.000000	0.200000	1.000000
19	Q20	discharge_medications	1	0.000000	0.000000	0.200000	1.000000	0.200000	1.000000

Figure 5. Retrieval scores from the project presentation. The chart summarizes the retrieval benchmark comparison and illustrates the central result: scoped vector search retrieves the right patient-specific evidence far more reliably than unscoped global search.

7.2 Generation Evaluation

Generation evaluation compares scoped RAG answers with no-RAG generation. The metrics capture different aspects of answer quality:

1. ROUGE measures word-level overlap between the generated answer and a reference chart passage. Higher ROUGE means the answer says more of the same content as the reference.
2. BERTScore measures semantic similarity rather than exact word overlap, so paraphrases and synonyms are rewarded more than in ROUGE.
3. Faithfulness is a project-specific rule-of-thumb check for whether answer sentences stay supported by the retrieved patient text.

These results suggest that grounding generation in retrieved patient context produces answers that better align with the source note than asking the model to answer without retrieved evidence. The presentation's main generation finding is that RAG clearly outperforms no-RAG on chart overlap and semantic similarity, while faithfulness is strong but not perfect.

Reference source: 20/20 queries used real chunk text

	ID	Ref source	RAG ROUGE-1	RAG ROUGE-2	RAG ROUGE-L	RAG BERTScore	RAG Faith	NR ROUGE-1	NR ROUGE-2	NR ROUGE-L	NR BERTScore	Flag
0	Q01	chunk	0.273000	0.200000	0.273000	0.914000	1.000000	0.000000	0.000000	0.000000	0.816000	✓
1	Q02	chunk	0.230000	0.152000	0.151000	0.830000	1.000000	0.022000	0.000000	0.014000	0.767000	✓
2	Q03	chunk	0.381000	0.350000	0.381000	0.923000	1.000000	0.000000	0.000000	0.000000	0.786000	✓
3	Q04	chunk	0.675000	0.419000	0.615000	0.875000	1.000000	0.076000	0.000000	0.061000	0.781000	✓
4	Q05	chunk	0.100000	0.000000	0.100000	0.839000	1.000000	0.000000	0.000000	0.000000	0.784000	✓
5	Q06	chunk	0.052000	0.006000	0.045000	0.784000	1.000000	0.019000	0.000000	0.013000	0.754000	✓
6	Q07	chunk	0.375000	0.286000	0.375000	0.883000	1.000000	0.000000	0.000000	0.000000	0.789000	✓
7	Q08	chunk	0.299000	0.215000	0.299000	0.845000	1.000000	0.037000	0.000000	0.037000	0.785000	✓
8	Q09	chunk	0.211000	0.000000	0.211000	0.839000	1.000000	0.000000	0.000000	0.000000	0.791000	✓
9	Q10	chunk	0.413000	0.205000	0.263000	0.872000	1.000000	0.065000	0.000000	0.036000	0.786000	✓
10	Q11	chunk	0.519000	0.480000	0.519000	0.916000	0.500000	0.093000	0.000000	0.093000	0.833000	✓
11	Q12	chunk	0.711000	0.605000	0.667000	0.897000	1.000000	0.000000	0.000000	0.000000	0.796000	✓
12	Q13	chunk	0.353000	0.000000	0.353000	0.860000	0.750000	0.000000	0.000000	0.000000	0.798000	✓
13	Q14	chunk	0.346000	0.240000	0.231000	0.894000	1.000000	0.000000	0.000000	0.000000	0.816000	✓
14	Q15	chunk	0.509000	0.340000	0.473000	0.890000	0.889000	0.000000	0.000000	0.000000	0.760000	✓
15	Q16	chunk	0.000000	0.000000	0.000000	0.833000	1.000000	0.000000	0.000000	0.000000	0.836000	✓
16	Q17	chunk	0.619000	0.311000	0.395000	0.878000	0.917000	0.040000	0.000000	0.016000	0.787000	✓
17	Q18	chunk	0.333000	0.130000	0.292000	0.894000	0.667000	0.000000	0.000000	0.000000	0.794000	✓
18	Q19	chunk	0.000000	0.000000	0.000000	0.812000	1.000000	0.000000	0.000000	0.000000	0.800000	✓
19	Q20	chunk	0.519000	0.458000	0.500000	0.869000	0.800000	0.049000	0.000000	0.025000	0.756000	✓

Figure 6. Generation scores from the project presentation

The chart compares answer quality metrics for RAG and no-RAG responses, showing stronger chart alignment when retrieved patient context is included.

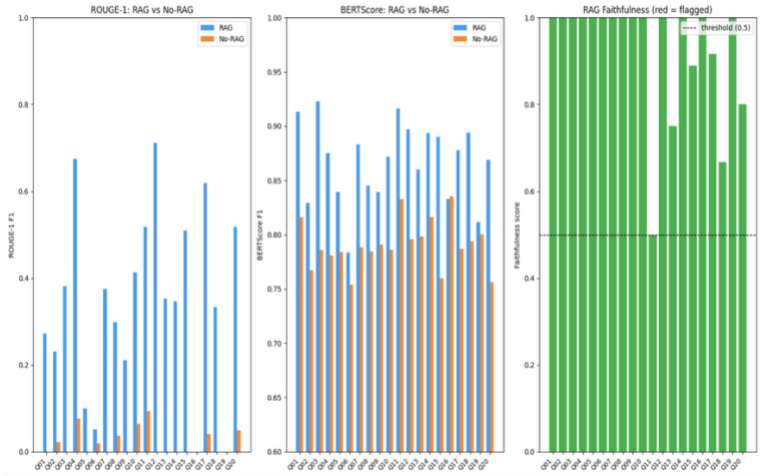

Figure 7. RAG versus no-RAG answer comparison

The presentation figure highlights the practical difference between grounded answers that use retrieved chart evidence and unsupported answers from a model without patient-specific context.

7.3 Notebook Walkthrough

`notebooks/CDSS_Evaluation_Walkthrough.ipynb` provides exploratory evaluation, visualizations, recall-at-k analysis, strategy comparison plots, and qualitative examples. It also documents known limitations such as short allergy chunks and the need for a larger evaluation set. The notebook and presentation are also the sources for the report diagrams above. These figures cover system architecture, retrieval depth selection, section-level recall behavior, strategy comparison, retrieval scores, and generation quality.

8. Testing

The repository contains unit tests for major backend modules:

- `tests/test_ingestion.py`
- `tests/test_ner.py`
- `tests/test_retrieval.py`
- `tests/test_generation.py`

These tests provide coverage around data loading behavior, entity extraction, retrieval logic, and generation behavior. The evaluation scripts provide an additional functional assessment of retrieval and answer quality.

9. Key Findings

The most important findings from the project are:

1. Patient-scoped retrieval is much more reliable than global vector search for clinical QA over many similar admissions.
2. Section-aware chunking improves interpretability because returned sources can be tied to clinical note sections.
3. Combining structured data with unstructured discharge notes makes the patient overview more useful than RAG alone.
4. Local inference with Ollama is feasible for a research/demo CDSS, although performance depends on local hardware.
5. Short or sparse sections, especially allergies, may be missed by embedding retrieval and may require structured extraction or special handling.
6. Simpler scoped retrieval was more valuable than advanced fusion methods at the current project scale.

For stakeholders, the main takeaway is practical: patient-specific scoping is foundational, local models plus vector search can produce measurable gains over LLM-only responses, and transparent source chunks are necessary for clinical trust.

10. Safety, Privacy, and Ethics

The project was designed around safety, privacy, and ethical use of clinical data.

10.1 Ethical Data Use

The system uses MIMIC-IV, which is a de-identified dataset released for credentialed research use [1]. The project does not require or redistribute raw identifiable patient data. Any use of the dataset still depends on PhysioNet credentialing, data-use agreements, and institutional rules.

10.2 Privacy

The default architecture is local-only. Embedding generation, retrieval, vector storage, and answer generation run on the user's machine through Ollama and ChromaDB. No patient text is sent to external LLM APIs in the default setup. This reduces privacy risk because clinical notes remain inside the local environment rather than moving through third-party inference services.

10.3 Security and Attack Surface

Because the default system does not call external model APIs, it avoids a major outbound data exposure path. The backend and vector database are intended for local or controlled-network deployment. For a production deployment, additional controls would still be required, including authentication, authorization, network isolation, audit logging, and secure configuration management.

10.4 Hallucination and Clinical Safety

The system is designed to minimize hallucinations by using RAG instead of free-form generation. Answers are generated from retrieved patient-specific context, and the frontend exposes source chunks so users can inspect the evidence. In evaluation, scoped RAG showed substantially better faithfulness than no-RAG generation. However, no clinical AI system should claim a permanent guarantee of zero hallucinations. The safer interpretation is that this design reduces unsupported answers by grounding responses in retrieved chart evidence.

11. Limitations

This project is a research and prototype system, not a production clinical product. Current limitations include:

- The evaluation set is small: 20 questions over 10 held-out patients.
- This benchmark is useful for development, but it is not large enough for statutory, regulatory, or clinical validation.
- The NER model uses broad clinical entity spans and section-based heuristics, so entity categories may be imperfect.
- Retrieval quality can degrade for very short sections or sparse text.

- Extremely short allergy sections are a known failure mode; structured allergy data would be safer for safety-critical allergy checks.
- The system does not yet implement production authentication, audit logging, role-based access, or PHI governance.
- ChromaDB and Ollama are local services, so deployment requires a host with persistent storage and suitable compute.
- The frontend can be deployed as a static application, but the backend is not suitable for serverless-only hosting in its current form.

12. Future Work

Recommended next steps include:

1. Expand the evaluation set to include more patients, more question types, and more clinical sections.
2. Add structured lookup paths for high-value fields such as allergies and discharge medications.
3. Evaluate stronger clinical NER models and compare them against the current SciSpaCy baseline.
4. Add authentication, audit trails, and access controls for a more realistic clinical deployment.
5. Improve retrieval with section routing, hybrid lexical/vector search, or reranking if validated against a larger test set.
6. Package the backend for deployment on a VM or container host with persistent ChromaDB storage and managed Ollama/model serving.

13. Conclusion

This CDSS project demonstrates a complete local RAG workflow for clinical discharge notes. It integrates MIMIC-IV structured and unstructured data, clinical section parsing, entity extraction, vector retrieval, grounded generation, an API layer, and a usable frontend dashboard.

The strongest technical result is the effectiveness of patient-scoped retrieval, which substantially improves recall compared with global vector search. The system also shows that a local-first architecture can support useful clinical QA workflows without relying on external LLM APIs. With a larger evaluation set, stronger governance features, and targeted improvements for sparse clinical fields, this prototype could serve as a strong foundation for further clinical NLP and decision-support research.

14. References

[1] A. E. W. Johnson et al., "MIMIC-IV, a freely accessible electronic health record dataset," ***Scientific Data***, vol. 10, article 1, 2023. https://doi.org/10.1038/s41597-022-01899-x

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," ***Advances in Neural Information Processing Systems***, vol. 33, 2020. https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[3] M. Neumann, D. King, I. Beltagy, and W. Ammar, "SciSpaCy: Fast and Robust Models for Biomedical Natural Language Processing," ***Proceedings of the 18th BioNLP Workshop and Shared Task***, 2019. https://doi.org/10.18653/v1/W19-5034

[4] Z. Rackauckas, "RAG-Fusion: A New Take on Retrieval Augmented Generation," ***International Journal on Natural Language Computing***, vol. 13, no. 1, pp. 37-47, 2024. https://doi.org/10.5121/ijnlc.2024.13103

[5] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods," ***Proceedings of SIGIR***, 2009. https://doi.org/10.1145/1571941.1572114